

GRAPHS — The Unified Expert Mental Model

How to THINK about graphs (Striver A2Z-aligned, interview-derivation focused)

*Goal of these notes: not to make you remember 20 algorithms, but to install **one operating system** in your head from which all 20 algorithms can be re-derived in an interview under pressure.*

PART 0 — THE GRAPH ECOSYSTEM MAP

Everything in graphs hangs off this tree. Memorize the *shape* of this tree, not the leaves.

GRAPHS

- 0. Representation (adj list / matrix / edge list) everything depends on this
- 1. Traversal the "engine room"; 70% of problems are disguised traversals
 - BFS (wave / layers) unweighted shortest path, multi-source, bipartite
 - DFS (tunnel / backtrack) components, cycles, topo sort, bridges, SCC
- 2. Connectivity
 - Connected Components (BFS/DFS loop over all nodes)
 - DSU (incremental / dynamic connectivity)
- 3. Cycles
 - Undirected: DFS + parent OR DSU
 - Directed: DFS + recursion stack OR Kahn's (topo fails cycle)
- 4. Ordering (DAGs only)
 - Topological Sort (DFS finish-time)
 - Kahn's Algorithm (BFS on indegree)
- 5. Shortest Paths
 - Unweighted BFS $O(V+E)$
 - Weighted, ≥ 0 Dijkstra $O(E \log V)$

Weighted, negative	Bellman-Ford	$O(V \cdot E)$
All pairs	Floyd-Warshall	$O(V^3)$
DAG	Topo sort + relax	$O(V+E)$ (works with negatives!)
6. MST (undirected, weighted, connect-everything-cheaply)		
	Prim's (grow a blob; Dijkstra's cousin)	
	Kruskal's (sort edges + DSU)	
7. Deep DFS structure (the "advanced" tier — all powered by DFS timestamps)		
	Bridges	
	Articulation Points	
	SCC Kosaraju (2-pass DFS) / Tarjan (1-pass, low-link)	
8. Applied patterns		
	Grid problems (implicit graphs)	
	Multi-source BFS (fire spreading from many points)	
	Graph DP (DP on DAGs / longest path / counting paths)	

Dependency arrows (what unlocks what)

Representation	BFS/DFS	Components, Cycles, Bipartite
DFS	Topo Sort	DAG Shortest Path, Graph DP
BFS	Multi-source BFS, 0-1 BFS, Dijkstra (BFS + priority queue)	
Dijkstra	Prim's (same skeleton, different key)	
DSU	Kruskal's, dynamic connectivity, cycle detection	
DFS timestamps	Bridges	Articulation Points Tarjan SCC

The single most important realization: there are really only **THREE engines** in all of graph theory:

1. **BFS** — process nodes in order of *distance/layers* (queue)
2. **DFS** — process nodes in order of *depth/finish-time* (stack/recursion)
3. **DSU** — process *edges* and merge groups (no traversal at all)

Every named algorithm is one of these engines plus **one extra piece of tracked information**:

Algorithm	Engine	The one extra idea
Connected Components	BFS/DFS	run from every unvisited node
Cycle (undirected)	DFS	remember your parent
Cycle (directed)	DFS	remember the current path (rec stack)

Bipartite	BFS/DFS	2-color instead of visited
Topo (DFS)	DFS	push on <i>finish</i> , reverse
Kahn's	BFS	queue keyed by indegree==0
Dijkstra	BFS	queue min-heap keyed by distance
0-1 BFS	BFS	deque (push front for 0-edges)
Bellman-Ford	none	brute-force relax all edges V-1 times
Prim's	Dijkstra	heap keyed by <i>edge weight</i> , not path length
Kruskal's	DSU	sort edges first
Bridges/AP/Tarjan	DFS	discovery time + low-link
Kosaraju	DFS	finish order + reversed graph
Multi-source BFS	BFS	seed queue with many nodes
Graph DP	Topo/DFS	dp[] relaxed in topological order

If you internalize this table, you can never “forget” an algorithm — you can only forget *one extra idea*, and one idea is recoverable.

PART 0.5 — THE UNIVERSAL QUESTIONS (ask these for EVERY algorithm)

Experts don't store code; they store answers to five questions:

1. **What information is tracked?** (visited? dist[]? parent[]? indegree[]? low[]?)
2. **In what ORDER are nodes/edges processed?** (FIFO? LIFO? by min-dist? by sorted weight? by finish time?)
3. **What is the INVARIANT?** (the sentence that stays true after every step and proves correctness)
4. **When is a value FINAL?** (BFS/Dijkstra: when popped. Bellman-Ford: only after V-1 rounds. DSU: never — it's live.)
5. **What breaks the invariant?** (negative edges break Dijkstra; cycles break topo sort; directedness breaks parent-trick cycle detection)

When you read each topic below, you'll see these five questions answered. In an interview, *re-deriving* = *re-answering these five questions for the problem at hand*.

PART 1 — GRAPH REPRESENTATION

Why it exists

A graph is a *relationship*, not a layout. The computer needs the relationship “who is connected to whom” in a queryable form. Before choosing algorithms, you must choose: **how do I ask ‘who are u’s neighbors?’ fast?**

Mental model

Think of a city’s phone directory:

- **Adjacency list** = each person keeps a contact list of friends. (“Who are *my* friends?” — instant.)
- **Adjacency matrix** = a giant $V \times V$ grid of “do i and j know each other?” (“Are A and B friends?” — instant, but listing A’s friends takes V checks.)
- **Edge list** = a pile of friendship certificates. (Great for sorting all friendships by ‘strength’ — hello, Kruskal’s.)

The three forms

```
// 1. Adjacency list (DEFAULT — use this 95% of the time)
vector<vector<int>>> adj(n);          // unweighted
vector<vector<pair<int,int>>>> adj(n); // weighted: {neighbor, weight}
adj[u].push_back(v);
adj[v].push_back(u);                // omit for directed graphs

// 2. Adjacency matrix (only when  $V \leq \sim 1000$ , or Floyd-Warshall)
vector<vector<int>>> mat(n, vector<int>(n, 0));
mat[u][v] = mat[v][u] = 1;

// 3. Edge list (Kruskal's, Bellman-Ford)
vector<array<int,3>> edges;          // {w, u, v}
```

Trade-off table (derive, don’t memorize)

	Adj List	Adj Matrix	Edge List
Space	$O(V+E)$	$O(V^2)$	$O(E)$
“neighbors of u?”	$O(\deg(u))$	$O(V)$	$O(E)$

"edge u-v exists?"	$O(\deg(u))$	$O(1)$	$O(E)$
Iterate all edges sorted	—	—	$O(E \log E)$

Why list wins by default: traversal algorithms ask "neighbors of u?" once per node total cost = $\sum \deg(u) = 2E = O(V+E)$. With a matrix the same loop costs $O(V^2)$ even for sparse graphs.

Hidden gotchas interviewers test

- **0-indexed vs 1-indexed** nodes (Striver problems are often 1-indexed: allocate `n+1`).
- **Undirected = add edge both ways.** Forgetting the reverse edge is the #1 silent bug.
- **Multi-edges and self-loops** — does your cycle detection survive a self-loop?
- **Disconnected graphs** — representation is fine, but *your loop over algorithms must start from every node.*
- **Grids are graphs** — you never build adj; neighbors are computed on the fly via direction arrays (see Part 8).

Relation to trees

A tree IS a graph: connected, acyclic, $E = V-1$. You probably stored trees as `Node*` with children pointers — that's just an adjacency list where edges point one way. Graphs generalize: edges can go anywhere, cycles can exist, the structure can be disconnected. **What transfers:** "visit children" becomes "visit neighbors." **What breaks:** in a tree you never revisit a node by accident; in a graph you will, hence the single most important new tool: `visited[]`.

10-second recall

Adjacency list by default · $O(V+E)$ space · push both directions if undirected · matrix only for dense $V \leq 10^3$ / Floyd-Warshall · edge list only for Kruskal/Bellman-Ford.

PART 2 — BFS (Breadth-First Search)

STEP 0 — Place in ecosystem

The first of the two traversal engines. Unlocks: shortest path in unweighted graphs, bipartite check, multi-source spreading, Kahn's, 0-1 BFS, and (with a heap swap) Dijkstra.

STEP 1 — Why was it invented?

Problem: "What is the minimum number of edges from S to every other node?"

DFS explores one deep tunnel at a time — it might reach a node via a 50-edge path when a 2-

edge path existed. We need an exploration order that **guarantees the first time we touch a node, we touched it via a shortest route**. Core idea in one sentence: **explore in expanding rings — all nodes at distance d before any node at distance $d+1$** .

STEP 2 — Mental model

Drop a stone in a pond. The ripple expands as a perfect circle. Every point on the pond gets wet **exactly when the ripple reaches it**, and the ring number = its distance from the stone. The queue is *the current ring's wavefront*. Close your eyes: a wave, sweeping outward, layer by layer, never skipping a layer.

STEP 3 — First-principles derivation

1. Naive: try all paths from S to T , take the shortest — exponential. Pain.
2. Observation: I don't need all paths — I need the *first arrival time* at each node, like a flood.
3. Simulate the flood: at time 0, S is wet. At time 1, all neighbors of S . At time $t+1$, all dry neighbors of time- t nodes.
4. Data structure that processes things in arrival order = **FIFO queue**.
5. To avoid wetting a node twice (and looping forever in a cycle) **visited[]**, marked **when enqueued**.

That's the entire algorithm. You just invented BFS.

STEP 4 — Information flow

- **Tracked:** `visited[]` (or `dist[]` doubling as `visited`), `dist[]`, optionally `parent[]` for path reconstruction, and the **queue = the frontier** (the boundary between explored and unexplored).
- **What changes:** queue contents shift from ring d to ring $d+1$; `dist` gets stamped once per node.
- **Invariant (the one to say out loud in interviews):** *the queue always contains nodes of at most two consecutive distance values, in non-decreasing distance order; when a node is dequeued, its `dist` is final.*
- **When final:** a node's distance is correct the moment it is **enqueued** (in plain BFS). It is never updated again — that's why BFS needs no "relaxation."

STEP 5 — Complete dry run

Graph (undirected), source = 0:

```
0 — 1   0 — 2
1 — 3   2 — 4
```

3 — 4 4 — 5

Step	Action	Queue	dist[0..5]	visited
init	enqueue 0	[0]	0 ∞ ∞ ∞ ∞ ∞	{0}
1	pop 0; see 1,2	[1,2]	0 1 1 ∞ ∞ ∞	{0,1,2}
2	pop 1; see 0 ,3	[2,3]	0 1 1 2 ∞ ∞	{0,1,2,3}
3	pop 2; see 0 ,4	[3,4]	0 1 1 2 2 ∞	{0,1,2,3,4}
4	pop 3; see 1 ,4	[4]	(no change)	
5	pop 4; see 2 3 ,5	[5]	0 1 1 2 2 3	all
6	pop 5; see 4	[]	done	

Notice the queue at every moment holds *consecutive layers only*: {1,2} then {2,3} then {3,4}...
The wave is visible in the data structure itself.

STEP 6 — Code derivation

```
vector<int> bfs(int src, vector<vector<int>>& adj, int n) {
    vector<int> dist(n, -1);    // -1   unvisited; doubles as visited[]
    queue<int> q;
    dist[src] = 0;              // mark BEFORE pushing — see mistake #1 below
    q.push(src);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) {
            if (dist[v] == -1) { // why check here, not after popping?
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
    return dist;
}
```

Line-by-line “why”:

- `dist[v]==-1` check **at enqueue time, not pop time**: in unweighted BFS the first discovery is already optimal, so re-adding a node is pure waste — worse, without it a node can enter the queue many times and on dense graphs the queue blows up. (In Dijkstra this changes — there, better paths can be found later, so we use lazy deletion. Knowing *this exact difference* is a senior-level signal.)

- $\text{dist}[v] = \text{dist}[u] + 1$ is the *only* update rule; it is correct **because** u was dequeued with final distance.
- Marking before pushing (not after popping) prevents the same node being enqueued twice by two different neighbors in the same layer.

STEP 7 — Complexity from first principles

Each node enters the queue **at most once** (guarded by visited) $\leq V$ pushes/pops. When u is popped we scan $\text{adj}[u]$ total edge scans = $\sum \text{deg}(u) = 2E$. Nothing else happens. Total = **$O(V + E)$** . It's not a formula — it's bookkeeping: V queue events + $2E$ neighbor glances.

STEP 8 — Common interview mistakes

- Marking visited at **pop** instead of **push** duplicates in queue TLE on dense cases (still “correct” output, which makes it sneaky).
- Forgetting disconnected graphs unreachable nodes keep $\text{dist} = -1$; if the problem asks “fill all,” loop the BFS over all unvisited starts.
- Using BFS for **weighted** shortest paths. The wave metaphor breaks: a weight-5 edge is not “one ripple step.” (Exception: weights 0/1 deque-based 0-1 BFS.)
- Path reconstruction: store $\text{parent}[v]=u$ at enqueue, walk back from target.
- Grids: forgetting bounds checks or walking into walls.

STEP 9 — Recognition signals

“minimum number of steps / moves / operations” · “shortest path, unweighted” · “fewest transformations (word ladder!)” · “level / layer / nearest” · “spread / infect / rot simultaneously” (multi-source) · state-space search where each move costs 1 (locks, puzzles).

False positives: “shortest path” **with weights** Dijkstra; “longest path” DP/DFS, never BFS; “all paths” backtracking DFS.

STEP 10 — Interview re-derivation script

Ask yourself: “Does every move cost the same?” yes “Then first-touch = optimal” “What processes things in first-come order? A queue.” “How do I not loop? visited at enqueue.” Done — 4 sentences and the algorithm exists again.

STEP 11 — Relation to trees

BFS on a tree = **level-order traversal** — you already know it! Transfers: queue mechanics, layer thinking. Breaks: trees have no cycles, so level-order never needed visited ; graphs do. Also, BFS from a source *induces a tree* (the BFS tree of parent pointers) — many graph algorithms are secretly “build a tree over the graph, reason on the tree.”

STEP 12 — Problem families

- Basics: BFS traversal (Striver), Number of Provinces.
- Shortest steps: Shortest Path in Binary Matrix (LC 1091), Word Ladder (LC 127), Knight's minimum moves.
- Spreading: Rotting Oranges (LC 994), 01 Matrix (LC 542) — multi-source.
- Layered logic: Bipartite check (LC 785), Snakes and Ladders (LC 909).
- Pattern: in all of them, "one move = cost 1" and "minimum" appear together.

STEP 13 — Expert insights

Strong candidates say "the queue holds at most two layers" unprompted; weak candidates can't explain *why* visited goes at enqueue. CP veterans see "minimum operations to transform X into Y" and instantly think *implicit graph: states = nodes, ops = edges, BFS*. The graph is often **not given** — you imagine it.

STEP 14 — 10-second recall sheet

1. Intuition: ripple in a pond.
 2. Picture: expanding rings.
 3. Tracked: queue (frontier), dist, visited-at-enqueue.
 4. Signal: "minimum steps, unit cost."
 5. Invariant: dequeue order = non-decreasing distance; first touch is final.
 6. $O(V+E)$.
 7. Mistake: visited at pop.
 8. Derivation: equal costs first arrival optimal FIFO.
-

PART 3 — DFS (Depth-First Search)

STEP 0 — Place in ecosystem

The second engine. BFS answers "how far?"; DFS answers questions about **structure**: components, cycles, ordering, bridges, SCCs. The entire "advanced" tier (Part 7) is DFS + timestamps.

STEP 1 — Why invented?

BFS is great for distance but clumsy for questions like "does a cycle exist?", "in what order do dependencies finish?", "is this edge critical?" Those need to understand the **shape of paths** — you need to walk a path to its end and *remember how you got there*. Core idea in one sentence: **go as deep as possible; backtrack only when stuck; the recursion stack IS the current path**.

STEP 2 — Mental model

You're exploring a cave with a ball of string (Theseus). At every junction, pick an untried tunnel and go deeper, unrolling string. Dead end (or all tunnels tried) follow string back one junction. The **string = recursion stack = the exact path from entrance to where you stand right now**. This "the stack is the current path" fact is the superpower BFS lacks — it's why DFS detects directed cycles, finds bridges, orders dependencies.

STEP 3 — First-principles derivation

1. Want: visit everything reachable. 2. Greedy idea: just keep walking to any new neighbor.
3. Problem: dead end — need to return to last junction with options need memory of the route **stack** (or recursion, which is a free stack). 4. Problem: cycles make you walk forever **visited[]**. That's DFS. The elegance: recursion gives you path-memory *for free*.

STEP 4 — Information flow

- **Tracked:** `visited[]` ; **implicitly** the recursion stack (current root here path); optionally entry/exit timestamps, parent, color states.
- **Three lifetimes of a node** (the expert's lens):
 - **WHITE** — untouched
 - **GRAY** — on the current path (entered, not yet exited) *this* is what BFS cannot express
 - **BLACK** — fully explored (exited)
- **Invariant:** at any instant, the GRAY nodes form a single path from the start to the current node.
- Edge classification (directed): edge $u \rightarrow v$ lands on WHITE = tree edge; GRAY = **back edge cycle**; BLACK = forward/cross edge. This single classification powers cycle detection, topo sort, and Tarjan.

STEP 5 — Dry run (with entry/exit times — they matter later)

Graph: 0 1, 0 2, 1 3, 3 2

```
call dfs(0): time=1, enter 0 (GRAY)      path: 0
call dfs(1): time=2, enter 1              path: 0-1
call dfs(3): time=3, enter 3              path: 0-1-3
  neighbor 2 WHITE  dfs(2): time=4        path: 0-1-3-2
  no new neighbors  exit 2, time=5 (BLACK)
  exit 3, time=6      path: 0-1
  exit 1, time=7      path: 0
  neighbor 2 is BLACK skip (cross edge, NOT a cycle!)
  exit 0, time=8
```

Note: 0 2 reaching a BLACK node is **not** a cycle. Only GRAY = cycle. Candidates who use plain visited (no GRAY/BLACK split) report a false cycle here. This is *the* classic directed-cycle trap.

STEP 6 — Code derivation

```
void dfs(int u, vector<vector<int>>& adj, vector<int>& vis) {
    vis[u] = 1;           // mark on ENTRY (= turning GRAY)
    for (int v : adj[u])
        if (!vis[v]) dfs(v, adj, vis); // recursion = free path stack
    // "exit point": code here runs when u turns BLACK.
    // Topo sort pushes u here. Tarjan computes low[u] here.
    // The post-order position is where DFS's real power lives.
}
```

- Mark on entry, or a cycle re-enters u infinitely.
- The blank line before return isn't decoration — *post-order is real estate*. Every advanced DFS algorithm is "DFS + something interesting at the exit point."
- Iterative version (explicit stack) exists for $V \geq \sim 10^5$ to dodge stack overflow — mention this in interviews, it scores points.

STEP 7 — Complexity

Each node entered once (visited guard) V calls; each call scans its adjacency $\sum \text{deg} = 2E$.
 $O(V+E)$, same accounting as BFS — the engines differ in *order*, never in cost.

STEP 8 — Common mistakes

- Using one visited array for directed-cycle detection (needs GRAY/BLACK — see dry run).
- Stack overflow on deep graphs (a path of 10^6 nodes) — switch to iterative.
- Confusing "visited" with "on current path" when counting paths or backtracking; backtracking problems must **unmark** on exit, traversal must not.
- Believing DFS finds shortest paths. It does not. Ever. (Unweighted shortest = BFS.)

STEP 9 — Recognition signals

"any path / does a path exist" · "all paths / enumerate" (backtracking-DFS) · "connected components" · "cycle?" · "ordering / dependency" (topo) · "flood fill region" · anything needing post-order or path memory.

False positive: "shortest/minimum" that's BFS/Dijkstra territory.

STEP 10 — Interview re-derivation

"Do I care about distance or structure?" Structure DFS. *"Do I need to know the current path?"* Yes recursion stack gives it free. *"Do I need to act when a node finishes?"* Yes write code after the loop (post-order).

STEP 11 — Relation to trees

DFS on a tree = pre/in/post-order traversal — you already know DFS! Tree recursion transfers 100%: "do something on entry (preorder), recurse, do something on exit (postorder)." What breaks: graphs revisit nodes (need visited), have multiple edge types (back/cross edges don't exist in trees), and may be disconnected (need the outer for-loop). A DFS run also induces a **DFS tree** — and "non-tree edges are back edges" (in undirected graphs) is the foundation of bridges/articulation points.

STEP 12 — Problem families

Flood fill (LC 733), Number of Islands (LC 200), Surrounded Regions (LC 130), All Paths From Source to Target (LC 797), Clone Graph (LC 133), cycle detection problems, and everything in Parts 4–7.

STEP 13 — Expert insight

Experts think in **colors and timestamps**, not just visited. The question "what do I know at this node's exit time?" generates topo sort, Tarjan, tree-DP and more. Weak candidates treat DFS as "BFS with a stack" — true for plain reachability, deeply false for everything structural: the *order* (LIFO depth-first, with well-defined finish times) is the entire point.

STEP 14 — Recall sheet

1. Intuition: cave + string. 2. Picture: tunneling deep, backtracking. 3. Tracked: visited, recursion stack = current path, (entry/exit times). 4. Signal: structure questions — existence, cycles, ordering, components. 5. Invariant: GRAY nodes = current path. 6. $O(V+E)$. 7. Mistake: no GRAY/BLACK in directed cycle check. 8. Derivation: need path memory stack recursion.
-

PART 4 — CONNECTED COMPONENTS

Why / mental model

Cities and roads: a component = one "island" of mutual reachability. Problem before: "are u, v in the same network?" — answering per-query with a fresh traversal is wasteful. Idea: **one**

traversal paints one entire island; loop over all nodes, start a new paint color whenever you find unpainted land.

The pattern (memorize this skeleton — it wraps EVERYTHING)

```
int components = 0;
for (int i = 0; i < n; i++)
    if (!vis[i]) { components++; dfs(i); } // or bfs(i)
```

This outer loop is not “the components algorithm” — it’s a **universal harness**. Cycle detection, bipartite, topo sort on disconnected graphs: all wrap in this same loop. Interviewers love dropping disconnected inputs to catch people who forgot the harness.

Information flow

Tracked: `vis[]` , optionally `comp[u] = id` (which island `u` belongs to). Invariant: after `dfs(i)` returns, *every* node reachable from `i` is marked. Hence each loop-triggered `dfs` = exactly one new component.

Complexity

The harness doesn’t multiply cost: every node is visited once across ALL `dfs` calls combined still **$O(V+E)$** total. (Beginners wrongly say $O(V \cdot (V+E))$.)

Recognition signals

“number of provinces/islands/networks” · “can everyone reach everyone?” · “minimum edges to connect all components” (answer: `components - 1`) · “groups of friends.”

For *incremental* connectivity (edges added over time, queries interleaved) **DSU instead** (Part 10).

Trees

A tree = exactly one component with $E = V - 1$. A forest = many components. Quick check that screams “expert”: a graph with V nodes and $< V - 1$ edges **cannot** be connected.

Recall

Outer loop over unvisited + any traversal · count starts · $O(V+E)$ · comp ids enable $O(1)$
same-island queries · dynamic version = DSU.

PART 5 — CYCLE DETECTION (the two worlds)

This topic is REALLY two different problems. Mixing them up is the most common cycle-detection failure in interviews.

5A. Undirected graphs

Mental model: walking in a park; if you arrive at a place you've already seen *and it isn't the place you just came from*, you've looped.

Derivation: in undirected DFS, every edge $u-v$ is seen twice ($u \rightarrow v$ and $v \rightarrow u$). Seeing your own parent again is not a cycle — it's the same edge. So: visited neighbor + not parent = a second distinct route to that node = cycle.

```
bool dfs(int u, int parent) {
    vis[u] = 1;
    for (int v : adj[u]) {
        if (!vis[v]) { if (dfs(v, u)) return true; }
        else if (v != parent) return true; // visited & not parent = cycle
    }
    return false;
}
```

Alternative — DSU: for each edge (u,v) : if $\text{find}(u) == \text{find}(v)$ they were *already* connected, so this edge closes a cycle. (This is literally Kruskal's rejection test.)

Alternative — BFS: same parent trick, store $\{\text{node}, \text{parent}\}$ in the queue.

Edge cases interviewers poke: parallel edges ($u-v$ twice = cycle; parent trick by *node* fails, track edge id), self-loop (instant cycle), disconnected graph (harness loop!).

5B. Directed graphs

Why the parent trick dies: in a directed graph, $u \rightarrow v$ existing says nothing about $v \rightarrow u$; "not parent" is meaningless. A cycle now means: an edge pointing to a node **on the current path** — a GRAY node.

Mental model: following one-way streets; a cycle = a one-way sign pointing back into the street sequence you are *currently driving*, not merely a street you drove yesterday.

```
// color: 0=WHITE 1=GRAY(on path) 2=BLACK(done)
bool dfs(int u) {
    color[u] = 1;
    for (int v : adj[u]) {
        if (color[v] == 1) return true;    // back edge    cycle
        if (color[v] == 0 && dfs(v)) return true;
    }
    color[u] = 2;    // exit: leave the path
    return false;
}
```

Invariant: GRAY set = exact current path. Edge into GRAY = cycle. Edge into BLACK = harmless (see Part 3 dry run).

Equivalent method: run Kahn's algorithm — if processed nodes $< V$, a cycle exists (cycle nodes never reach indegree 0). Use Kahn's when you'd anyway need the ordering, or to avoid recursion depth.

Recognition signals

"deadlock detection" · "can all courses be finished?" (LC 207 = directed cycle!) · "is this a valid tree?" (= connected + no cycle, LC 261) · "redundant connection" (LC 684 — DSU cycle).

Relation to trees

Tree = the acyclic ideal. Undirected: $|E| \geq V$ = cycle guaranteed (pigeonhole — another instant expert check). Directed: no such shortcut; you must look for back edges.

Recall

Undirected: visited && not-parent (or DSU union fails). Directed: GRAY hit on recursion stack (or Kahn count $< V$). NEVER use the parent trick on directed graphs.

PART 6 — BIPARTITE GRAPHS

Why invented

Real problems split entities into two camps with edges only *across* camps: workers = jobs, students = projects, "is this network 2-colorable?" Question: can we color every node red/blue so that no edge joins same colors?

Mental model

Two rival teams shaking hands — handshakes only between teams. Equivalent deep fact you should *derive once and own forever*: **bipartite** **no odd-length cycle**. Why: walking a cycle alternates colors red,blue,red,blue...; returning to start needs an even number of steps. One odd cycle = contradiction = not bipartite.

Derivation

Coloring has no real choice: pick a start, color it 0; every neighbor is *forced* to 1; their neighbors forced to 0... So just BFS/DFS, assigning `color[v] = 1 - color[u]`. Conflict (edge between same colors) = odd cycle = not bipartite. No conflict anywhere = the 2-coloring in your hands is the proof.

```
bool bfsCheck(int src) {
    queue<int> q; q.push(src); color[src] = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) {
            if (color[v] == -1) { color[v] = 1 - color[u]; q.push(v); }
            else if (color[v] == color[u]) return false;
        }
    }
    return true;
}
// harness: run from every uncolored node (disconnected graphs!)
```

Information flow: `color[]` replaces `visited[]` (`-1` = unvisited). Invariant: all colored nodes so far are consistent. $O(V+E)$.

Mistakes & signals

Mistakes: forgetting the harness; assuming connectivity; trying to be clever about *which* color (the first node's color is arbitrary — components are independent).

Signals: “divide into two groups” · “enemies/rivals can't be together” (LC 886 Possible Bipartition) · “alternating” · odd-cycle language. False positive: splitting into two groups *of equal size* — that's partitioning/DP, not bipartiteness.

Recall

2-color via BFS/DFS, forced moves only · conflict = odd cycle = not bipartite · color[] doubles as visited · $O(V+E)$ · run on every component.

PART 7 — TOPOLOGICAL SORT & KAHN'S ALGORITHM

STEP 0 — Ecosystem

Lives only in **DAGs** (directed acyclic graphs). Depends on: DFS (finish times) or BFS (Kahn's).

Unlocks: DAG shortest/longest paths, graph DP, build systems, course scheduling. Topo sort is *the* bridge between graphs and DP.

STEP 1 — Why invented

You must take courses with prerequisites, compile files with dependencies, get dressed (socks before shoes). Need a **linear order where every arrow points forward** — every prerequisite appears before its dependent. Impossible iff there's a cycle (A needs B needs A). Core idea: **flatten a partial order into a total order.**

STEP 2 — Mental model

Two pictures, one per algorithm:

- **Kahn's:** a to-do list where you may only do tasks whose prerequisites are all done. Each morning, do everything currently unblocked; that unblocks new things. (Bonus: tasks done on the same "morning" can run in parallel — interviewers love this extension.)
- **DFS version:** to truly "finish" a task you must first finish everything it leads to. A node's **finish time** is when its whole downstream world is complete — later finishers must come earlier in the order — **reverse finish order = topo order.**

STEP 3 — Derivation (Kahn's, the more "inventable" one)

1. Who can go first? A node with no prerequisites = **indegree 0**. (A DAG always has one — walk arrows backwards; no cycles — you must stop somewhere.)
2. Output it. Its outgoing edges are now satisfied — *delete them* — some indegrees drop to 0 — new candidates.
3. Repeat. "Delete edges" is implemented as decrementing an `indegree[]` array; "candidates" sit in a queue.
4. If the queue dies before all V are output — leftovers all have $\text{indegree} \geq 1$ — they point at each other — **cycle detected for free.**

STEP 4 — Information flow

Kahn's tracks: `indegree[]` (the only state!), queue of ready nodes, output list. Invariant: *a node enters the queue exactly when its last prerequisite is processed* — output order respects all edges.
 DFS version tracks: visited + an output stack pushed at **exit time**; invariant: when u finishes, all of u's descendants already finished (are below it in the stack).

STEP 5 — Dry run (Kahn's)

Edges: 5 0, 5 2, 4 0, 4 1, 2 3, 3 1
 indegree: 0:2 1:2 2:1 3:1 4:0 5:0

```
Queue [4,5]      output []
pop 4  dec 0( 1), 1( 1) output [4]   queue [5]
pop 5  dec 0( 0!), 2( 0!) output [4,5] queue [0,2]
pop 0  no out-edges   output [4,5,0] queue [2]
pop 2  dec 3( 0!)     output [4,5,0,2] queue [3]
pop 3  dec 1( 0!)     output [...,3] queue [1]
pop 1  output [4,5,0,2,3,1] count==V no cycle
```

Watch the *exact* moment each node enters: precisely when its counter hits 0 — when its last blocker clears.

STEP 6 — Code

```
vector<int> kahn(int n, vector<vector<int>>>& adj) {
    vector<int> indeg(n, 0), topo;
    for (int u = 0; u < n; u++) for (int v : adj[u]) indeg[v]++;
    queue<int> q;
    for (int i = 0; i < n; i++) if (indeg[i] == 0) q.push(i); // ALL sources
    while (!q.empty()) {
        int u = q.front(); q.pop();
        topo.push_back(u);
        for (int v : adj[u])
            if (--indeg[v] == 0) q.push(v); // last prerequisite just cleared
    }
    if ((int)topo.size() < n) return {}; // cycle!
    return topo;
}
```

DFS version: plain DFS; `st.push(u)` after the neighbor loop; pop everything at the end. **Must include the GRAY check if a cycle may exist** — otherwise it outputs garbage silently.

STEP 7 — Complexity

Indegree build touches each edge once: $O(E)$. Each node enqueued once: $O(V)$. Each edge decremented once: $O(E)$. **$O(V+E)$** — every line of work maps to one node or one edge, nothing repeats.

STEP 8 — Mistakes

- Running topo on a graph **with** a cycle and not checking `topo.size() < n` (Kahn) or GRAY states (DFS).
- DFS version: pushing at *entry* instead of *exit* (produces a wrong, plausible-looking order — brutal to debug).
- Forgetting multiple valid answers exist; if the problem wants lexicographically smallest swap queue for a **min-heap** (classic follow-up).
- Modeling edges backwards: “a is prerequisite of b” = edge a → b. Read the statement twice.

STEP 9 — Recognition

“prerequisites” · “build/compile order” · “task scheduling with dependencies” · “alien dictionary — derive letter order” (LC 269: each adjacent word pair gives one edge!) · “safe states / eventual safe nodes” · any DP whose states form a DAG.

False positive: dependencies *with* possible cycles where you must output something anyway that’s SCC condensation first (Part 12).

STEP 10 — Interview re-derivation

“Who has no blockers? Start there. What changes after I finish it? Its dependents lose one blocker. What structure hands me ‘newly unblocked’ items? A queue.” — you just rebuilt Kahn’s in three questions.

STEP 11 — Trees

A rooted tree is a DAG; “parents before children” (any BFS/preorder) is a valid topo order. Transfers: post-order intuition — DFS-topo *is* post-order, reversed. Breaks: trees have one root; DAGs may have many indegree-0 sources and many valid orders.

STEP 12 — Problems

Course Schedule I/II (LC 207/210 — cycle check / actual order), Alien Dictionary (LC 269), Eventual Safe States (LC 802 — reverse edges + Kahn), Minimum Height Trees (LC 310 — Kahn-style leaf peeling), Longest Path in DAG (topo + DP), Parallel Courses (LC 1136 — “mornings” = layers).

STEP 13 — Expert insight

Experts see topo sort as “**the order in which DP on a DAG must be evaluated**” — that reframing converts dozens of hard problems into easy ones (Part 15). They also know Kahn’s layer count = minimum parallel rounds, and that the topo order being *unique* a Hamiltonian path exists in the DAG (queue size stays ≤ 1).

STEP 14 — Recall

1. Intuition: do unblocked tasks first.
2. Picture: dependency arrows all pointing right.
3. Tracked: indegree[] + ready-queue (or DFS finish stack).
4. Signal: “prerequisite / order / dependency”.
5. Invariant: node output only after all in-edges consumed.
6. $O(V+E)$.
7. Mistake: no cycle check.
8. Derivation: indegree-0 must exist in a DAG — peel repeatedly.

PART 8 — SHORTEST PATHS: THE DECISION TREE

Before the three named algorithms, install the **dispatcher**. In an interview, choosing correctly IS half the answer:

Weights all equal (or unweighted)?	BFS	$O(V+E)$
Weights only 0 and 1?	0-1 BFS (deque)	$O(V+E)$
Non-negative weights, one source?	Dijkstra	$O(E \log V)$
Negative weights possible, one source?	Bellman-Ford	$O(V \cdot E)$
Need to DETECT negative cycle?	Bellman-Ford (Vth pass)	
Graph is a DAG (any weights, even neg)?	Topo sort + relax	$O(V+E)$
All pairs, V small ($\leq \sim 400$)?	Floyd-Warshall	$O(V^3)$
"Longest path"?	general graphs: NP-hard; DAG: negate/DP	

The shared atom — RELAXATION. Every weighted shortest-path algorithm is built from one operation:

```
if (dist[u] + w < dist[v]) dist[v] = dist[u] + w; // "found a better route to v through u"
```

The algorithms differ ONLY in **what order, and how many times, edges are relaxed**:

- Dijkstra: relax edges out of nodes in increasing dist order — each node once (greedy, needs $w \geq 0$).
- Bellman-Ford: relax ALL edges, $V-1$ rounds (brute force, survives negatives).
- DAG: relax in topo order — each edge once (order is handed to you for free).

- Floyd-Warshall: relax via intermediate vertices, k-outermost (DP over allowed midpoints).

Hold onto this: **one atom, four schedules.**

PART 9 — DIJKSTRA

STEP 1 — Why invented

BFS dies on weights: “fewest edges” $\neq$ “least total weight” (a 2-edge path of weight 100 loses to a 5-edge path of weight 10). We need the wave idea back, but where the wave moves through **cost-space, not edge-count-space**.

STEP 2 — Mental model

Fire spreading along ropes, where rope length = weight, and fire moves at constant speed. The fire reaches every junction **exactly at time = its shortest distance**. Alternative picture: a growing blob of “settled territory” — nodes whose distance is provably final — that always annexes the *nearest* unsettled node next.

STEP 3 — Derivation (invent it yourself)

1. Try BFS with weights — fails: queue order no longer equals distance order.
2. Fix the order: don’t process in FIFO order, process **whichever frontier node currently has the smallest tentative distance** — exactly like the fire reaching junctions in time order.
3. Data structure that yields the minimum: **min-heap**. So Dijkstra = *BFS with the queue upgraded to a priority queue*.
4. ☒ $\geq d$ (u was the minimum!), and travel onward through **non-negative** edges, so its total $\geq d$.
5. That proof step “edges can’t subtract” is exactly where **negative weights kill Dijkstra** — a route through a farther node could shrink. Knowing *where the proof uses the assumption* is the difference between memorizing and understanding.

STEP 4 — Information flow

Tracked: `dist[]` (best-known-so-far), min-heap of $\{\text{dist}, \text{node}\}$ = frontier with priorities, optionally `parent[]`. Invariant: **popped nodes have final distances; heap holds tentative ones**. Key contrast with BFS: in BFS a node’s `dist` is final at *enqueue*; in Dijkstra only at *pop* — between push and pop it can improve, which is why a node may sit in the heap multiple times (lazy deletion):

```
if (d > dist[u]) continue; // stale heap entry — the node was already settled cheaper
```

STEP 5 — Dry run

Edges: 0-1(4), 0-2(1), 2-1(2), 1-3(1), 2-3(5) source 0

```
heap {(0,0)}          dist [0, ∞, ∞, ∞]
pop (0,0) relax: 1 4 push(4,1); 2 1 push(1,2)
                        dist [0, 4, 1, ∞] heap {(1,2),(4,1)}
pop (1,2) relax: 1: 1+2=3 < 4 push(3,1); 3: 1+5=6 push(6,3)
                        dist [0, 3, 1, 6] heap {(3,1),(4,1),(6,3)}
pop (3,1) relax: 3: 3+1=4 < 6 push(4,3)
                        dist [0, 3, 1, 4] heap {(4,1),(4,3),(6,3)}
pop (4,1) 4 > dist[1]=3 STALE, skip      lazy deletion in action
pop (4,3) relax: nothing improves
pop (6,3) stale, skip. Final dist [0, 3, 1, 4]
```

Note node 1 entered the heap twice — first via the direct heavy edge, then via the cheaper detour. The heap *ordering* guaranteed the cheap version popped first.

STEP 6 — Code

```
vector<int> dijkstra(int src, int n, vector<vector<pair<int,int>>>& adj) {
    vector<int> dist(n, INT_MAX);
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq; // MIN-heap!
    dist[src] = 0; pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue; // stale entry guard
        for (auto [v, w] : adj[u])
            if (dist[u] + w < dist[v]) { // relax
                dist[v] = dist[u] + w;
                pq.push({dist[v], v}); // push, even if v already inside
            }
    }
    return dist;
}
```

Why-lines: pair ordered {dist, node} so the heap compares by dist; `greater<>` flips C++'s default max-heap (forgetting this is the #1 Dijkstra bug); the stale guard substitutes for

“decrease-key,” which std heaps don’t support.

STEP 7 — Complexity from first principles

Each edge relaxation can push once $\leq E$ heap entries pushes/pops cost $\log(\text{heap size}) \leq \log E \approx \log V$ (since $E \leq V^2$, $\log E \leq 2 \log V$). Total: **$O(E \log V)$** . The log is the *price of always knowing the minimum*; BFS gets order for free because all steps cost 1.

STEP 8 — Mistakes

- Max-heap by default in C++ (silent wrong answers).
- Running it with negative edges — may output wrong dists *without crashing*. Say the proof-break reason out loud.
- Missing stale-entry guard correct but can TLE.
- int overflow: dist sums up to $V \cdot \text{maxW}$ — use `long long` when weights are big.
- “Visited” array marking at *push* (BFS habit) blocks the cheaper later route **WRONG ANSWERS**. If you mark, mark at pop.

STEP 9 — Recognition

“minimum cost/time/effort path” with non-negative weights · “network delay time” (LC 743) · “cheapest” with per-edge costs · grids where cells have costs (Path With Minimum Effort LC 1631 — minimize the max edge: Dijkstra still works because min-of-max is monotone along the pop order) · implicit state graphs with weighted moves.

“at most K stops” (LC 787) — the constraint breaks the “popped = final” invariant; use Bellman-Ford with K rounds or BFS-on-(node,stops) states. Negative weights. “longest path.”

STEP 10 — Interview re-derivation

“BFS but weighted process by smallest tentative distance min-heap when popped, can anything beat it? Not if edges are ≥ 0 settle on pop, relax neighbors.” Five clauses, full algorithm.

STEP 11 — Trees

On a tree there’s exactly ONE path between any pair — shortest path = DFS/BFS to find it; Dijkstra is overkill. The “settled blob with parent pointers” Dijkstra builds is the **shortest-path tree** — output of Dijkstra is literally a tree over the graph. Comparing it with the *minimum spanning tree* (different objective!) is a classic conceptual interview question: SPT minimizes dist-from-source per node; MST minimizes total edge sum. They generally differ.

STEP 14 — Recall

1. Intuition: fire along weighted ropes. 2. Picture: growing settled blob, nearest node annexed next. 3. Tracked: `dist[]`, min-heap frontier. 4. Signal: min cost path, weights ≥ 0 .
 5. Invariant: pop final (proof needs $w \geq 0$). 6. $O(E \log V)$. 7. Mistakes: max-heap default, negatives, visited-at-push. 8. Derivation: BFS + priority queue + greedy proof.
-

PART 10 — BELLMAN-FORD

Why invented

Negative edges are real (currency arbitrage, potentials, costs with rebates) and they break Dijkstra's greedy proof. Need something that doesn't *commit* early. Core idea: **stop being clever — relax every edge, again and again, until nothing changes.**

Mental model

Rumor spreading in rounds. In round k , the truth ("best price") has provably propagated along all cheapest paths that use $\leq k$ edges. A shortest *simple* path has $\leq V-1$ edges (more edges repeated node contains a cycle, and removing a non-negative cycle never hurts) **$V-1$ rounds suffice.**

Information flow + invariant

Tracked: `dist[]` only. **Invariant after round k : `dist[v] ≤ length of the best path using  $\leq k$  edges`.** This invariant is the entire algorithm — it dictates the round count and gives the negative-cycle test: if a **V -th round still improves something**, some best path "wants" $\geq V$ edges — it loops a node profitably **negative cycle reachable from source**. Detection isn't a bolt-on trick; it falls out of the invariant.

```
vector<int> bellman(int n, vector<array<int,3>>& edges, int src) { // {u,v,w}
    vector<int> dist(n, 1e9); dist[src] = 0;
    for (int round = 1; round <= n - 1; round++)
        for (auto& [u, v, w] : edges)
            if (dist[u] != (int)1e9 && dist[u] + w < dist[v]) // guard: don't relax from ∞!
                dist[v] = dist[u] + w;
    for (auto& [u, v, w] : edges) // V-th pass
        if (dist[u] != (int)1e9 && dist[u] + w < dist[v]) { /* negative cycle */ }
    return dist;
}
```


Complexity

$V-1$ rounds $\times$ E relaxations = $O(V \cdot E)$. Felt from the invariant: each round buys “one more edge of path length,” and paths max out at $V-1$ edges. Optimization worth mentioning: stop early when a full round makes no change.

Mistakes & signals

Mistakes: relaxing from ∞ (the guard above — classic WA); thinking it *computes* distances inside a negative cycle (those are $-\infty$, it only *detects*); confusing “negative edge” (fine) with “negative cycle” (distances undefined).

Signals: “negative weights” $\cdot$ “detect arbitrage” (log-prices: product > 1 $\rightarrow$ negative cycle of $-\log$) $\cdot$ “at most K stops/edges” (run exactly K rounds — LC 787 Cheapest Flights, with the per-round copy-of-dist trick so each round adds ≤ 1 edge) $\cdot$ small $V \cdot E$ budgets.

False positive: non-negative weights with tight constraints — Dijkstra dominates.

Recall

Relax all E edges, $V-1$ rounds $\cdot$ invariant: round k = best $\leq k$ -edge paths $\cdot$ V -th round improves $\rightarrow$ negative cycle $\cdot O(VE) \cdot$ guard against $\infty \cdot K$ -rounds variant = “at most K edges.”

PART 11 — FLOYD-WARSHALL

Why invented

All-pairs shortest paths. Running Dijkstra from every node = $O(V \cdot E \log V)$ and fails on negatives; Bellman-Ford from every node = $O(V^2 E)$. Want one clean sweep. Core idea: **DP over allowed intermediate vertices**.

Mental model & derivation

$dp[k][i][j]$ = shortest $i \rightarrow j$ path allowed to pass through only nodes $\{0 \dots k\}$ as intermediates.

Adding node k as a permitted midpoint, every pair (i, j) has just two choices:

- ignore k : $dp[k-1][i][j]$
- route through k : $dp[k-1][i][k] + dp[k-1][k][j]$ (each half uses only $< k$ midpoints — no double counting)

```
for (int k = 0; k < n; k++)      // K IS THE OUTERMOST LOOP. Always.
  for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
```

```
if (dist[i][k] + dist[k][j] < dist[i][j])
    dist[i][j] = dist[i][k] + dist[k][j];
```

The k-dimension compresses away because round k only reads round k-1 values of the forms $[i][k]$ and $[k][j]$, and those don't change *during* round k (relaxing via k can't improve a path to k via k).

Facts to own

- Handles negative **edges**; $\text{dist}[i][i] < 0$ after the run — negative cycle through i — free detection.
- Init: $\text{dist}[i][i]=0$, edge weights where edges exist, ∞ elsewhere (beware overflow: check $\text{dist}[i][k] < \text{INF}$ before adding, or use `1e9` with ints... and still guard).
- $O(V^3)$ time, $O(V^2)$ space — fine for $V \leq \sim 400$.
- Same DP skeleton computes **transitive closure** (reachability, with OR/AND) and **min-of-max / max-of-min** path problems — the algebra is swappable.

Mistakes & signals

Mistakes: k as the inner loop (subtly wrong answers — the proof above *requires* k outermost); ∞ + weight overflow.

Signals: “distance between every pair” · “queries: $\text{dist}(u,v)$, many of them, small V” · “find the city with fewest reachable neighbors within threshold” (LC 1334). $V = 10^5$ — V^3 is fantasy; rethink.

Recall

DP on allowed midpoints · k outermost · $O(V^3)$ · negatives OK, $\text{diag} < 0$ — neg cycle · $V \leq 400$.

PART 12 — DSU (DISJOINT SET UNION / UNION-FIND)

STEP 0 — Ecosystem

The third engine — the only one that never traverses. Powers: Kruskal's, dynamic connectivity, cycle detection by edges, “accounts merge”-style grouping. Processes a **stream of edges**, maintaining “who's in which group” at all times.

STEP 1 — Why invented

BFS/DFS answer connectivity for a *frozen* graph. But many problems are **incremental**: edges arrive one by one, with “are u,v connected *now*?” queries interleaved. Re-running BFS per query = $O(Q \cdot (V+E))$ — dead. Need a structure where union and find are near- $O(1)$. Core idea: **each group elects a representative; connectivity = same representative.**

STEP 2 — Mental model

Companies merging. Every employee can ask up the management chain “who is our CEO?” Two people are in the same company — same CEO. A merger = one CEO starts reporting to the other. Two famous optimizations in this metaphor:

- **Path compression**: after asking up the chain once, everyone you passed re-wires to report **directly to the CEO** — next query is instant.
- **Union by size/rank**: in a merger, the **smaller** company’s CEO reports to the bigger one’s — keeps chains short.

STEP 3 — Derivation

1. Naive: store group id per node; union = relabel a whole group = $O(V)$ per union. Pain.
2. Lazy idea: don’t relabel — just link one leader under another (a forest of trees). Union $O(1)$! But find = walk to root = $O(\text{chain length})$, can degrade to $O(V)$.
3. Fix chains while you walk them (compression) + never hang big under small (by size). Together: amortized **$O(\alpha(N))$** , α = inverse Ackermann ≤ 4 for any N in this universe. “Effectively constant.”

STEP 4 — Information flow

Tracked: `parent[]` (the forest), `size[]` or `rank[]`. Invariant: **`find(u) == find(v)` u,v are in the same component, at all times.** The trees’ shapes are meaningless; only the root identity matters — this is why compression is free to flatten aggressively.

STEP 5 — Dry run

```
parent: [0,1,2,3,4] size:[1,1,1,1,1]
union(0,1): roots 0,1; equal size 1 under 0. parent [0,0,2,3,4] size[0]=2
union(2,3): parent [0,0,2,2,4] size[2]=2
union(1,3): find(1)=0, find(3)=2; sizes 2,2 2 under 0
           parent [0,0,0,2,4] size[0]=4
find(3): 3 2 0; compress: parent[3]=0 parent [0,0,0,0,4]
union(3,4): find(3)=0 (1 hop now!), find(4)=4; 4 under 0. All connected.
edge (1,4) arrives: find(1)==find(4)==0 would create a CYCLE reject (Kruskal's heart)
```

STEP 6 — Code (the canonical class — this one IS worth having at your fingertips)

```
struct DSU {
    vector<int> p, sz;
    DSU(int n): p(n), sz(n, 1) { iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); } // path compression
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false; // already same group — cycle edge!
        if (sz[a] < sz[b]) swap(a, b); // union by size
        p[b] = a; sz[a] += sz[b];
        return true;
    }
};
```

✓ = `find(p[x])` does compression in one elegant line — every node on the path re-points to the root on the way back. Returning `bool` from `unite` gives you cycle detection and “useful edge?” for free.

STEP 7 — Complexity

Without optimizations: $O(V)$ worst find. By-size alone tree height $\leq \log V$ (each time a node's root changes, its component at least doubles — can happen $\leq \log V$ times). Adding compression amortized $\alpha(N)$. For interviews, say: “*amortized inverse-Ackermann, effectively constant; $\log V$ from union-by-size alone.*”

STEP 8 — Mistakes

- Comparing `parent[u] == parent[v]` instead of `find(u) == find(v)` — parents go stale; only roots are truth.
- Forgetting to union by size *or* compress — usually still passes, but know which guarantee you dropped.
- Trying to use DSU for **directed** connectivity — DSU is symmetric-only (that's SCC's job).
- Trying to **delete** edges — DSU can't un-merge. Trick: process offline, **reverse time**, deletions become insertions (classic hard-problem move).
- Mapping non-integer items (emails, coordinates) — hash to ids first.

STEP 9 — Recognition

Edges arriving over time + connectivity queries · “earliest moment everyone is connected” (LC 1101) · “redundant connection” (LC 684) · “accounts merge” (LC 721) · “number of islands II

— land appears one cell at a time” (LC 305) · “satisfiability of equality equations” (LC 990) · Kruskal’s · “swap-groups” problems (indices that can be swapped form groups; sort within groups — LC 1202).

Single static graph, one-shot components plain BFS/DFS is simpler. Directed reachability.

STEP 10 — Interview re-derivation

“I need groups that merge. Give each group a leader. Same leader = same group. Merge = link leaders. Walks too long? Flatten as I walk; hang small under big.” That’s the entire structure from zero.

STEP 11 — Trees

DSU literally *stores* components as trees — but inverted (child → parent pointers, no child lists) and shape-irrelevant. Tree intuition that transfers: height arguments (the $\log V$ bound). Tree intuition that breaks: these trees aren’t traversed, aren’t ordered, and are deliberately flattened toward stars.

STEP 14 — Recall

1. Intuition: merging companies, ask-the-CEO.
 2. Picture: forests flattening into stars.
 3. Tracked: `parent[]`, `size[]`.
 4. Signal: incremental edges + group queries.
 5. Invariant: same root → same component.
 6. $\alpha(N)$ amortized.
 7. Mistake: comparing parents, not roots.
 8. Derivation: leaders + link + flatten.
-

PART 13 — MST: PRIM’S & KRUSKAL’S

STEP 0/1 — Why

Connect all cities with roads at minimum total cost. Tree is forced: a cycle means a removable edge → any optimal answer has $V-1$ edges, no cycles → spanning **tree**. The question is only *which* $V-1$ edges.

The one theorem powering both (cut property)

For any cut (partition of nodes into S and $V \setminus S$), the minimum-weight edge crossing the cut is safe to include in some MST. Exchange argument: if an MST skipped it, adding it creates a cycle; that cycle must cross the cut somewhere else with weight $\geq$ ours; swap them — cost didn’t increase. Both algorithms are just *strategies for repeatedly applying this theorem*:

- **Prim's:** the cut is always (my-blob, rest). Take the cheapest crossing edge, grow the blob. *One crystal growing.*
- **Kruskal's:** sort all edges; the cheapest remaining edge that joins two different components is the min edge over the cut separating those components. *Many droplets merging.*

Prim's — Dijkstra's twin

```
// EXACTLY Dijkstra's skeleton with ONE change:
// Dijkstra heap key: dist[u] + w (total path length from source)
// Prim's heap key: w (just the edge — cost to ATTACH v to the blob)
priority_queue<...> pq; // {edgeWeight, node}
// pop cheapest attach-cost node; if already inMST, skip; else add weight to answer,
// mark inMST, push all edges to non-MST neighbors keyed by w alone.
```

Information tracked: inMST[], heap keyed by attachment cost. Invariant: popped & fresh that edge is the min crossing edge of the current cut safe. Complexity: same accounting as Dijkstra **$O(E \log V)$** . Mistake: writing dist[u]+w out of Dijkstra muscle memory — know the one-line diff and you know both algorithms.

Kruskal's — sorting + DSU

```
sort(edges.begin(), edges.end()); // by weight
int cost = 0, used = 0;
for (auto& [w, u, v] : edges)
    if (dsu.unite(u, v)) { cost += w; if (++used == n - 1) break; }
```

Information tracked: DSU components; nothing else. Invariant: every accepted edge is the global-min edge joining two distinct components (cut property on that pair of components). Rejected edge = would form a cycle. Complexity: **$O(E \log E)$** for the sort; DSU ops are noise.

Choosing between them

Edge list given / sparse / DSU already needed — Kruskal. Adjacency list / dense / “grow from a node” feel — Prim. Both correct always (connected graphs).

Recognition & classics

“minimum cost to connect all points/cities/computers” (LC 1584 — build edges from coordinates!) · “min cost to supply water” (LC 1168 — virtual node trick: a well = edge to node 0) · “critical and pseudo-critical MST edges” (LC 1489). “minimum cost path A → B” — that’s

shortest path, not MST. Directed graphs — MST is undirected-only (directed version = arborescence, out of placement scope; just *name* it if asked).

Trees & recall

MST's output IS a tree — the cheapest skeleton of the graph. SPT (Dijkstra) $\neq$ MST: different objectives.

Cut property · greedy works · Prim = Dijkstra with $key=w$, $O(E \log V)$ · Kruskal = sort + DSU, $O(E \log E)$ · output has $V-1$ edges · “connect everything cheaply.”

PART 14 — BRIDGES & ARTICULATION POINTS

STEP 1 — Why

Networks fail. Which single cable, if cut, splits the internet (bridge)? Which single router, if down, disconnects users (articulation point)? Naive: remove each edge/node, re-run DFS $O(E \cdot (V+E)) / O(V \cdot (V+E))$. We want ONE DFS.

STEP 2 — Mental model

Run DFS; it lays a **tree** over the graph; all remaining edges are **back edges** = emergency escape ropes to ancestors (undirected graphs have no cross edges — key fact!). A tree edge $u-v$ is a **bridge** · v 's entire subtree has **no rope climbing above v** : cut the edge and the subtree is stranded. Everything reduces to: “*how high can this subtree escape?*”

STEP 3/4 — The two timestamps (the whole trick)

- $tin[u]$ — discovery time (when DFS entered u)
- $low[u]$ — the **minimum tin reachable** from u 's subtree using subtree tree-edges + at most the ropes (back edges) hanging off it.

Computed in **post-order** (DFS's exit-time real estate again!):

```
low[u] = min( tin[u],
             low[child]  for each tree-child,
             tin[neighbor] for each back edge u-neighbor (neighbor  $\neq$  parent) )
```

Bridge condition: edge $u-v$ ($v = \text{child}$) is a bridge $\text{low}[v] > \text{tin}[u]$ — v 's subtree cannot reach u or higher.

Articulation condition: u is an AP some child v has $\text{low}[v] \geq \text{tin}[u]$ (subtree can't escape *strictly above* u — reaching u itself doesn't help if u dies). **Root exception:** the DFS root has no ancestors; it's an AP it has ≥ 2 tree children.

Why $>$ for bridges but $\geq$ for APs? Cutting the *edge* still leaves u alive — reaching u (tin equal) saves you. Removing the *node* u kills it — reaching only u is worthless. Deriving this distinction live in an interview is a flex; do it.

Mini dry run

Graph: 0-1, 1-2, 2-0 (triangle), 1-3, 3-4
DFS from 0: tin 0:1, 1:2, 2:3, 3:4(via 1), 4:5
back edge 2-0 $\text{low}[2] = \min(3, \text{tin}[0]=1) = 1$ propagates: $\text{low}[1]=1$
node 4: leaf, $\text{low}=5$. Edge 3-4: $\text{low}[4]=5 > \text{tin}[3]=4$ BRIDGE
node 3: $\text{low}=4$. Edge 1-3: $\text{low}[3]=4 > \text{tin}[1]=2$ BRIDGE
edge 1-2 in triangle: $\text{low}[2]=1 \leq \text{tin}[1]=2$ not a bridge (the rope 2-0 saves it)
AP: node 1 (child 3 has $\text{low } 4 \geq \text{tin}[1] 2$) and node 3 (child 4). Triangle nodes safe.

Mistakes & recognition

- Using $\text{low}[\text{child}]$ for back edges instead of $\text{tin}[\text{neighbor}]$ — for plain bridges it happens to still work, but it breaks articulation points and is conceptually wrong (a back edge gives you the *ancestor's discovery time*, not its low).
- Forgetting the root's special rule.
- Parent handling with parallel edges: skip parent by *edge id*, not node id.
- Recursion depth on $V=10^5$.

Recognition: "critical connections" (LC 1192 — verbatim bridges) · "single point of failure" · "which edge removals disconnect" · "2-edge-connected components."

Constraint smell: must be one-pass, V up to 10^5 .

Recall

One DFS + tin/low · low = highest escape via subtree+one back edge · bridge: $\text{low}[v] > \text{tin}[u]$
· AP: $\text{low}[v] \geq \text{tin}[u]$, root needs 2 children · $O(V+E)$ · post-order computation.

PART 15 — SCC: KOSARAJU & TARJAN

STEP 1 — Why

In **directed** graphs, “connected” splits into a weaker and stronger notion. Strongly connected = mutually reachable ($u \rightarrow v$ AND $v \rightarrow u$). SCCs = maximal mutual-reachability clubs. Why care:

condensing each SCC to a super-node leaves a DAG — and DAGs unlock topo sort + DP. SCC is the tool that turns “directed graph with cycles” into “DAG,” after which the easy machinery applies. (LC 1192-style problems, 2-SAT, “reach everyone from one node?” all live here.)

STEP 2 — Mental model

One-way street city. An SCC = a neighborhood where you can drive between any two points both ways (roundabout cluster). Between neighborhoods, traffic flows one way only — the condensation DAG. Picture: blobs with one-way arrows between blobs.

Kosaraju — the 2-pass magic

Recipe: (1) DFS anywhere, record nodes by **finish time** (push on exit). (2) **Reverse every edge**. (3) Pop nodes in decreasing finish time; DFS each unvisited one **on the reversed graph**; each DFS = exactly one SCC.

Why it works (the part everyone hand-waves — don't): the latest finisher lies in a **source SCC** of the condensation (a DFS that touches an SCC can only finish after everything reachable from it — so source SCCs finish last). In the **reversed** graph, that source becomes a **sink**: DFS from it can't leak into other SCCs — reversed edges all point *into* it. So the first reverse-DFS carves out exactly one SCC; induction does the rest. One sentence to own: **reverse-finish-order + reversed graph = peel SCCs from the source side, leak-proof.**

```
// pass 1: dfs all, push u to stack at EXIT
// build reversed adjacency
// pass 2: while stack: pop u; if unvisited   dfs on reversed graph, collect one SCC
```

Complexity: two DFS sweeps + edge reversal = **$O(V+E)$** .

Tarjan — one pass with low-link

Same tin/low idea as bridges, adapted to directed graphs: keep an explicit stack of the current “open” nodes; `low[u]` = smallest tin reachable within the current open region. When `low[u] == tin[u]` at u's exit, u is the **root of an SCC**: pop the stack down to u — that popped set is one complete SCC. Extra care vs. bridges: only consider back edges to nodes **still on the stack** (cross edges to closed SCCs must be ignored — that's what the `onStack[]` flag is for). For placements: derive and code **Kosaraju** (easier to reproduce under pressure), *explain* Tarjan's idea fluently.

Mistakes & recognition

Mistakes: pushing at entry instead of exit in pass 1 (everything silently breaks); forgetting to reverse the graph; Tarjan without the onStack check.

Recognition: “mutually reachable groups” · “minimum edges/nodes to add so everything is reachable” (condense · count sources/sinks; answer = max(sources, sinks) on the condensation) · “2-SAT” · cycles in directed graphs where you need *which* nodes form them, not just existence.

Relation to everything: SCC = (DFS finish times from topo-sort thinking) + (low-link from bridges thinking). It’s the capstone — if you can re-derive Kosaraju’s correctness, your DFS understanding is interview-complete.

Recall

SCC = mutual reachability clubs · condensation = DAG · Kosaraju: finish order + reversed graph, 2 passes, $O(V+E)$ · why: last finisher · source SCC, reversal turns it into a leak-proof sink · Tarjan: tin/low + explicit stack, pop when low==tin.

PART 16 — GRID PROBLEMS (implicit graphs)

The reframe (this is the whole topic)

A grid IS a graph you never build: **cell = node, adjacency = computed on demand** via direction arrays. Once you see this, every grid problem collapses into a Part 1–9 problem wearing a costume.

```
int dr[] = {-1, 1, 0, 0};
int dc[] = {0, 0, -1, 1};      // add 4 diagonal pairs for 8-direction problems
for (int k = 0; k < 4; k++) {
    int nr = r + dr[k], nc = c + dc[k];
    if (nr < 0 || nr >= m || nc < 0 || nc >= n) continue; // bounds FIRST
    if (grid[nr][nc] == WALL || vis[nr][nc]) continue;
    ...
}
```

Translation table

Grid phrasing	Real algorithm
“count islands / regions”	connected components (DFS flood fill)

"shortest path, unit moves"	BFS, dist[][]
"rotting/fire/water spreads"	multi-source BFS
"min cost path, weighted cells"	Dijkstra on the grid
"can break $\leq K$ walls"	BFS on state (r, c, walls_used) — extra state dimension!
"knight moves"	BFS with 8 custom directions
"trapped / enclosed regions"	flood fill from the border , complement (LC 130)

The state-expansion idea (what separates medium from hard)

When a constraint travels with you (keys held, walls broken, steps mod k, direction faced), the node is no longer (r,c) — it's **(r,c,constraint)**. Visited becomes 3-D. "Shortest Path with K eliminations" (LC 1293) and "Shortest Path to Get All Keys" (LC 864, state = (r,c,key-bitmask)) are pure BFS once the state is right. Experts spend their thinking time on *defining the node*, then run a stock engine.

Mistakes

Bounds check order (index before checking = UB) · mutating grid as visited (fine & memory-savvy — but say it out loud, interviewer may want input untouched) · 4 vs 8 directions misread · m,n row/col swaps.

Recall

*Cell=node, dr/dc=edges, bounds first · pick engine by the table · constraint travels with you
widen the state · grids are graphs in costume.*

PART 17 — MULTI-SOURCE BFS

Why & model

"Every rotten orange rots neighbors each minute — when is everything rotten?" Running BFS from each source separately and taking minimums = $O(S \cdot (V+E))$. The flood insight: many stones dropped in the pond **simultaneously** — ripples merge; each cell's wet-time = distance to its **nearest** source. Implementation delta from plain BFS: **one line** — seed the queue with ALL sources at distance 0 before the loop. Everything else identical, still $O(V+E)$.

Equivalent mental trick: imagine a **virtual super-source** connected by 0-weight edges to all real sources; multi-source BFS = plain BFS from it. (The virtual-node trick generalizes: Dijkstra from many sources, MST with wells — LC 1168.)

Inverse-thinking applications (the real interview gold)

- “For each cell, distance to nearest 0” (LC 542): don’t BFS from each cell *to* a 0 — BFS **from all 0s outward**. Reverse the direction of the question.
- “Nearest exit,” “highest cell density,” “as far from land as possible” (LC 1162): same inversion.
- Rotting Oranges (LC 994): seed all rotten; answer = max dist; check for never-reached fresh cells.

Recall

Seed queue with all sources at dist 0 · = BFS from a virtual super-source · answers “distance to NEAREST source” for every node at once · invert “each X to nearest Y” into “all Y outward” · $O(V+E)$.

PART 18 — GRAPH DP (DP on DAGs)

Why & the central theorem

DP needs an evaluation order with no circular dependencies — i.e., **DP states + transitions must form a DAG**, and the valid evaluation order is exactly a **topological order**. Two directions of the same coin:

1. Graph problems on DAGs (longest path, path counting) are solved by DP in topo order.
2. Every DP you’ve ever written was secretly a DAG; “what order do I fill the table?” = “topo-sort the dependency graph.” (That’s why DP on general cyclic graphs fails, and why “longest path” is NP-hard in general but trivial in DAGs.)

The pattern

```
auto order = topoSort(adj);           // Kahn's or DFS
vector<long long> dp(n, BASE);
for (int u : order)                   // process in dependency order
    for (auto [v, w] : adj[u])
        dp[v] = combine(dp[v], dp[u] + w); // relax: max, min, +, count...
```

Swap combine :

- **Longest path in DAG:** max, BASE = 0 (or $-\infty$ for fixed source) — note Dijkstra can't do longest, but topo order can, negatives and all, because order is structural, not greedy.
- **Count paths s t:** $dp[s]=1$; $dp[v] += dp[u]$.
- **Shortest path in DAG with negatives:** min — beats Bellman-Ford at $O(V+E)$.
- **Memoized-DFS form:** $f(u)$ = combine over children $f(v)$ + memo — the recursion is implicit topo order. Cycle in graph infinite recursion either prove DAG or detect cycles first.

Recognition

“longest increasing path in matrix” (LC 329 — edges cell bigger-neighbor form a DAG because values strictly increase memoized DFS) · “number of ways to reach” with one-way moves · “longest chain/stack” (envelopes, boxes) · scheduling with profits on a dependency DAG · “eventual safe states.” Constraint smell: directed + acyclic (or acyclicity derivable from monotonicity: strictly increasing values, coordinates only growing, time only moving forward — monotone quantity DAG DP is legal).

Cycles present and unremovable not DP; consider SCC-condensation first (condense, then DP on the condensation DAG — a beautiful hard-problem combo).

Recall

DP order = topo order · DAG longest path, counting, min/max all become one relaxation sweep · monotone quantity implicit DAG · memoized DFS = topo in disguise · cycles condense via SCC first.

PART 19 — MASTER PATTERN-RECOGNITION TABLE

Read the problem statement; scan for these triggers:

Trigger phrase / structure	Reach for	Watch out
“minimum steps/moves”, unit cost	BFS	weighted? Dijkstra
“shortest path”, weights ≥ 0	Dijkstra	“at most K stops” breaks it Bellman/BFS-states
negative weights / “arbitrage”	Bellman-Ford	neg cycle $-\infty$ dists, detect only

"every pair", $V \leq 400$	Floyd-Warshall	k loop outermost
"prerequisite / order / dependency"	Topo / Kahn's	always cycle-check
"can finish all courses?"	directed cycle detection	= Kahn count < V
"two groups / no same-group edges"	bipartite 2-color	disconnected harness
"number of islands/provinces/groups"	components (DFS/BFS)	grid? flood fill
edges arrive over time + queries	DSU	can't delete; reverse time
"connect all, min total cost"	MST (Kruskal/Prim)	path A B $\neq$ MST
"critical connection / single failure"	bridges / artic. points	tin/low, root rule
"mutually reachable", directed	SCC + condensation	then topo/DP on DAG
"spreads simultaneously / nearest X"	multi-source BFS	invert the question
"longest path / count paths", one-way	DAG DP via topo	general longest = NP-hard
"fewest transformations between states"	implicit-graph BFS	define node = full state
keys/walls/fuel travel with you	BFS/Dijkstra on (pos, state)	widen visited dims

Meta-signals experts use:

- Constraints whisper the complexity: $V, E \leq 10^5$ $O(V+E)$ or $O(E \log V)$ only; $V \leq 400$ V^3 allowed (Floyd); $E \leq 10^4$ with K mentioned Bellman rounds.
 - The word "minimum" + uniform action BFS; + weighted action Dijkstra; + connect-everything MST; + ordering greedy/topo. Same word, four algorithms — the *object being minimized* decides.
 - No graph in sight? Ask "what are my states, what are my moves?" — word ladders, lock dials, water jugs, currency, course schedules are all graphs in disguise. **Inventing the graph is the actual skill being tested.**
-

PART 20 — THE INTERVIEW DERIVATION

PLAYBOOK (when memory fails)

A reusable script — works for any graph problem, even half-forgotten algorithms:

1. **Name the graph.** “Nodes are ___, edges are ___, directed? weighted?” Say it aloud; interviewers award points for explicit modeling.
 2. **Name the question type.** Distance? Structure (cycle/order/components)? Optimization over edges (MST)? Grouping (DSU/SCC)?
 3. **Pick the engine.** Distance+unit BFS. Structure DFS. Incremental groups DSU. Distance+weights BFS-with-a-heap (you can *re-invent* Dijkstra live with that phrase).
 4. **State the tracked information + invariant.** “I maintain `dist[]`; once popped, final, because edges are non-negative.” Interviewers literally probe for this sentence.
 5. **Edge-case sweep before coding:** disconnected? self-loops? 0 or 1 nodes? cycle possible? overflow? 1-indexed? This 20-second ritual prevents 80% of bugs.
 6. **Complexity by accounting,** not recitation: “each node enqueued once, each edge scanned twice $O(V+E)$.”
 7. If stuck: **degrade gracefully** — “brute force is re-running DFS per query at $O(Q \cdot (V+E))$; to beat it I need incremental connectivity that’s the shape DSU fills.” Naming the inefficiency usually *names the data structure*.
-

PART 21 — TREES GRAPHS: THE COMPLETE TRANSFER MAP

Tree concept you know	Graph generalization	What breaks
Level-order traversal	BFS	cycles need visited
Pre/post-order DFS	graph DFS, tin/tout times	revisits, back/cross edges, disconnection
One unique path between nodes	many paths “shortest” is now a real question	path uniqueness gone
Parent pointer	<code>parent[]</code> in BFS/DFS trees; DSU <code>parent</code>	DSU trees are shape-meaningless
Recursion on children	recursion on unvisited neighbors	must guard with visited

Tree DP (post-order combine)	DAG DP in topo order; low-link computations	needs acyclicity or SCC-condensation
Root	source(s); DAGs may have many indegree-0 roots	no canonical root
$E = V - 1$, connected, acyclic	the <i>definition</i> of a spanning tree; MST = cheapest one	general graphs violate all three
"Subtree"	DFS-subtree + escape ropes (back edges) bridges/APs	subtrees can escape upward

One-line summary to keep: **graph algorithms = tree algorithms + visited[] + a choice of processing order + an invariant that survives cycles.**

PART 22 — STRIVER A2Z / LEETCODE

PROBLEM LADDER (pattern-tagged)

Tier 1 — engines (do these until boring):

BFS/DFS traversal · Number of Provinces (LC 547, components) · Number of Islands (LC 200, grid flood fill) · Flood Fill (LC 733).

Tier 2 — one-extra-idea each:

Rotting Oranges (LC 994, multi-source) · 01 Matrix (LC 542, inverted multi-source) · Cycle in Undirected (parent trick + DSU version) · Bipartite (LC 785) · Course Schedule I/II (LC 207/210, directed cycle + Kahn) · Surrounded Regions (LC 130, border inversion) · Number of Enclaves (LC 1020, same inversion).

Tier 3 — weighted world:

Shortest Path in Binary Matrix (LC 1091) · Network Delay Time (LC 743, Dijkstra) · Path With Minimum Effort (LC 1631, Dijkstra on min-of-max) · Cheapest Flights Within K Stops (LC 787, Bellman-rounds — *the* Dijkstra false positive) · City With Smallest Number of Reachable Neighbours (LC 1334, Floyd) · Word Ladder I (LC 127, implicit graph BFS).

Tier 4 — structures:

Redundant Connection (LC 684, DSU cycle) · Accounts Merge (LC 721, DSU + hashing) · Most Stones Removed (LC 947, DSU components) · Min Cost to Connect All Points (LC 1584, MST on built edges) · Critical Connections (LC 1192, bridges) · Making a Large Island (LC 827, components + 1-flip enumeration).

Tier 5 — capstones:

Alien Dictionary (LC 269, build graph from word pairs + topo) · Longest Increasing Path in a

Matrix (LC 329, implicit DAG DP) · Shortest Path to Get All Keys (LC 864, state-expanded BFS) ·
 Eventual Safe States (LC 802, reverse-graph Kahn) · Kosaraju on a generic directed graph ·
 Swim in Rising Water (LC 778, Dijkstra/binary-search+BFS — two valid lenses, articulate both).

Rule of progression: **never move up a tier until you can re-derive (not recall) the tier below from its invariant.**

PART 23 — THE GRAND RECALL SHEET

(everything on one screen)

Topic	Picture	Tracks	Invariant	Final when	Complexity	#1 bug
BFS	pond ripple	queue, dist, vis	queue = ≤ 2 consecutive layers	at enqueue	$O(V+E)$	vis at pop
DFS	cave + string	vis, rec-stack	GRAY set = current path	at exit	$O(V+E)$	no GRAY/BLACK directed
Components	painting islands	vis, comp id	one outer-loop dfs = one comp	—	$O(V+E)$	forgetting harness
Cycle (undir)	re-met stranger	vis, parent	visited $\neg$ parent cycle	—	$O(V+E)$	parent trick on directed
Cycle (dir)	one-way loop	3 colors	edge GRAY cycle	—	$O(V+E)$	one visited array
Bipartite	two teams	color[]	forced alternating colors	—	$O(V+E)$	disconnected
Topo/Kahn	unblocked to-dos	indegree, queue	enqueue last prereq done	at output	$O(V+E)$	no cycle check
Dijkstra	fire on ropes	dist, min-heap	pop final ($w \geq 0$)	at pop	$O(E \log V)$	max-heap / negatives
Bellman-Ford	rumor rounds	dist	round k = best $\leq k$ -edge paths	after $V-1$ rounds	$O(VE)$	relaxing from ∞

Floyd-Warshall	allowed midpoints	dist[][]	k rounds = midpoints $\leq k$	after k loop	$O(V^3)$	k not outermost
DSU	merging companies	parent, size	same root same comp	live, always	$\alpha(N)$	comparing parents
Prim	growing crystal	inMST, heap(w)	pop = min cut edge	at pop	$O(E \log V)$	dist+w habit
Kruskal	droplets merging	DSU	accept joins 2 comps	$V-1$ edges	$O(E \log E)$	unsorted edges
Bridges/AP	escape ropes	tin, low	low = best subtree escape	post-order	$O(V+E)$	low vs tin on back edge
Kosaraju	peel source blobs	finish stack	rev-finish + rev-graph leak-proof	pass 2	$O(V+E)$	push at entry
Multi-src BFS	many stones, one pond	seeded queue	dist = nearest source	at enqueue	$O(V+E)$	not inverting question
DAG DP	conveyor of states	dp[], topo order	deps done before you	in topo order	$O(V+E)$	cycle unproven

The five questions, one last time

For any graph algorithm — known, forgotten, or brand new — interrogate it:

1) What's tracked? 2) In what order? 3) What invariant holds? 4) When is a value final? 5) What assumption breaks it?

Answer those five and you don't *remember* graph algorithms. You *own* them.